The type errors of our ways

Olivier Danvy*^ Risa Shindo* Quang-Trung Ta^

*Yale-NUS College, National University of Singapore
^School of Computing, National University of Singapore

IFL'19 - Singapore - September 27th, 2019

What is a type system?

- A type system is a set of rules that assigns types to constructs (expressions, statements) of a computer program.

[con] $A \vdash c : \mathrm{Typ}(c)$

[id] $A \vdash x : A(x)$

[abs] $$\frac{A[x : t_1] \vdash e : t_2}{A \vdash \texttt{fn}\ x \Rightarrow e : t_1 \rightarrow t_2}$$

[app] $$\frac{A \vdash e_1 : t_2 \rightarrow t_1 \quad A \vdash e_2 : t_2}{A \vdash e_1\ e_2 : t_1}$$

[let] $$\frac{A \vdash e_1 : ts_1 \quad A[x : ts_1] \vdash e_2 : t_2}{A \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 : t_2}$$

[rec] $$\frac{A[f : t] \vdash \texttt{fn}\ x \Rightarrow e : t}{A \vdash \texttt{rec}\ f\ x \Rightarrow e : t}$$

[if] $$\frac{A \vdash e_0 : \texttt{bool} \quad A \vdash e_1 : t \quad A \vdash e_2 : t}{A \vdash \texttt{if}\ e_0\ \texttt{then}\ e_1\ \texttt{else}\ e_2 : t}$$

[ins] $$\frac{A \vdash e : \forall \vec{\alpha}.\, t}{A \vdash e : S\,t} \quad \text{if } Dom(S) \subseteq \{\vec{\alpha}\}$$

[gen] $$\frac{A \vdash e : t}{A \vdash e : \forall \vec{\alpha}.\, t} \quad \text{if } \{\vec{\alpha}\} \cap FV(A) = \emptyset$$

Hindley-Milner type system
(Type and Effect Systems: Behaviours for Concurrency - Amtoft, Nielson, Nielson)

When is a type system used?

- To check or infer type of a given program.
- Type checking or type inference can happen:
  - Statically at compile time, like in ML, Haskell.
  - Or dynamically at run time, like in Scheme/Lisp, Python.
  - Or both statically and dynamically, like in recent gradual type systems.

Property of a type system

- In principle, a type system specifies the most general type of any given term, if it has one.
- In theory, a type system is a declarative logical construct:
  - It tells the type of a term if this term is well-typed.
  - However, it does not convey anything meaningful for ill-typed terms.

$$[\text{app}] \quad \frac{A \vdash e_1 : t_2 \to t_1 \quad A \vdash e_2 : t_2}{A \vdash e_1\, e_2 : t_1}$$

$$[\text{let}] \quad \frac{A \vdash e_1 : ts_1 \quad A[x : ts_1] \vdash e_2 : t_2}{A \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 : t_2}$$

Property of a type system

- In practice, type systems are operationalized into implementations:
  - Ill-typed terms elicit type errors.
  - Type errors depend on the traversal order over sub-terms.

$$[\text{rec}] \quad \frac{A[f : t] \vdash \texttt{fn}\ x \Rightarrow e : t}{A \vdash \texttt{rec}\ f\ x \Rightarrow e : t}$$

$$[\text{if}] \quad \frac{A \vdash e_0 : \texttt{bool} \quad A \vdash e_1 : t \quad A \vdash e_2 : t}{A \vdash \texttt{if}\ e_0\ \texttt{then}\ e_1\ \texttt{else}\ e_2 : t}$$

Type errors the OCaml way

Example 1: with integer expressions.

- Sometimes, type inferencers report errors that are very intelligible:

```
1
2 (* program successor1.ml *)
3
4 let f x =
5   (y, succ x y)
6
```

```
trungtq@bb8 $ ocamlc successor1.ml
File "successor1.ml", line 5, characters 3-4:
5 |   (y, succ x y)
      ^
Error: Unbound value y
```

Type errors the OCaml way

- But slight program changes can lead to type errors that are strange…

```
(* program successor1.ml *)

let f x =
  (y, succ x y)
```

```
(* program successor2.ml *)

let f x =
  (succ y x, y)
```

```
trungtq@bb8 $ ocamlc successor2.ml
File "successor2.ml", line 5, characters 3-7:
5 |   (succ y x, y)
       ^^^^
Error: This function has type int -> int
       It is applied to too many arguments; maybe you forgot a `;'.
```

Type errors the OCaml way

Example 2: with boolean expressions.

- Again, sometimes, type errors reported can be very clear:

```
1
2 (* program bool1.ml *)
3
4 let b =
5   (f false, not f true)
6
```

```
trungtq@bb8 $ ocamlc bool1.ml
File "bool1.ml", line 5, characters 3-4:
5 |   (f false, not f true)
        ^
Error: Unbound value f
```

Type errors the OCaml way

- Also, when the program is slightly changed, the reported type errors will be difficult to understand…

```
(* program bool1.ml *)

let b =
  (f false, not f true)
```

```
(* program bool2.ml *)

let b =
  (not f true, f false)
```

```
trungtq@bb8 $ ocamlc bool2.ml
File "bool2.ml", line 5, characters 3-6:
5 |   (not f true, f false)
       ^^^
Error: This function has type bool -> bool
       It is applied to too many arguments; maybe you forgot a `;'.
```

Type errors the OCaml way

Example 3: with string expressions.

- Again, sometimes, type errors can be intuitive:

```
1
2 (* program string1.ml *)
3
4 let f g z t =
5   (z ^ "a", z == "a", g (z "a"))
6
```

```
trungtq@bb8 $ ocamlc string1.ml
File "string1.ml", line 5, characters 25-26:
5 |   (z ^ "a", z == "a", g (z "a"))
                            ^
Error: This expression has type string
       This is not a function; it cannot be applied.
```

Type errors the OCaml way

- But when the program is slightly changed, the reported type errors can be unexpected…

```ocaml
(* program string1.ml *)

let f g z t =
  (z ^ "a", z == "a", g (z "a"))
```

```ocaml
(* program string2.ml *)

let f g z t =
  (g (z "a"), z ^ "a", z == "a")
```

```
trungtq@bb8 $ ocamlc string2.ml
File "string2.ml", line 5, characters 14-15:
5 |   (g (z "a"), z ^ "a", z == "a")
                 ^
Error: This expression has type string -> 'a
       but an expression was expected of type string
```

Type errors the OCaml way

- In type inferencers, an error message only reflects a particular traversal strategy when processing sub-terms (for OCaml, it is from left to right).
- Another strategy to traverse a given term is likely to elicit another error message in another context [McAdam 2002, Repairing Type Errors in Functional Programs, Ph.D. Thesis].
- But there is no taxonomy of type errors in general, so we cannot predict them.

Our contribution

- To parameterize the implementation of a type inferencer with its traversal strategy:
  - More than one traversal strategy can be used over ill-typed terms: left to right, right to left, random order.
  - A wider variety of error messages can be emitted: reporting multiple errors at the same time, on demand.

Our contribution

- Integrating to OCaml's compiler: consider traversal order of three types of expressions:
  - Tuple expression:

```
let t = (e1, e2, e3)
in …
```

  - If-then-else expression:

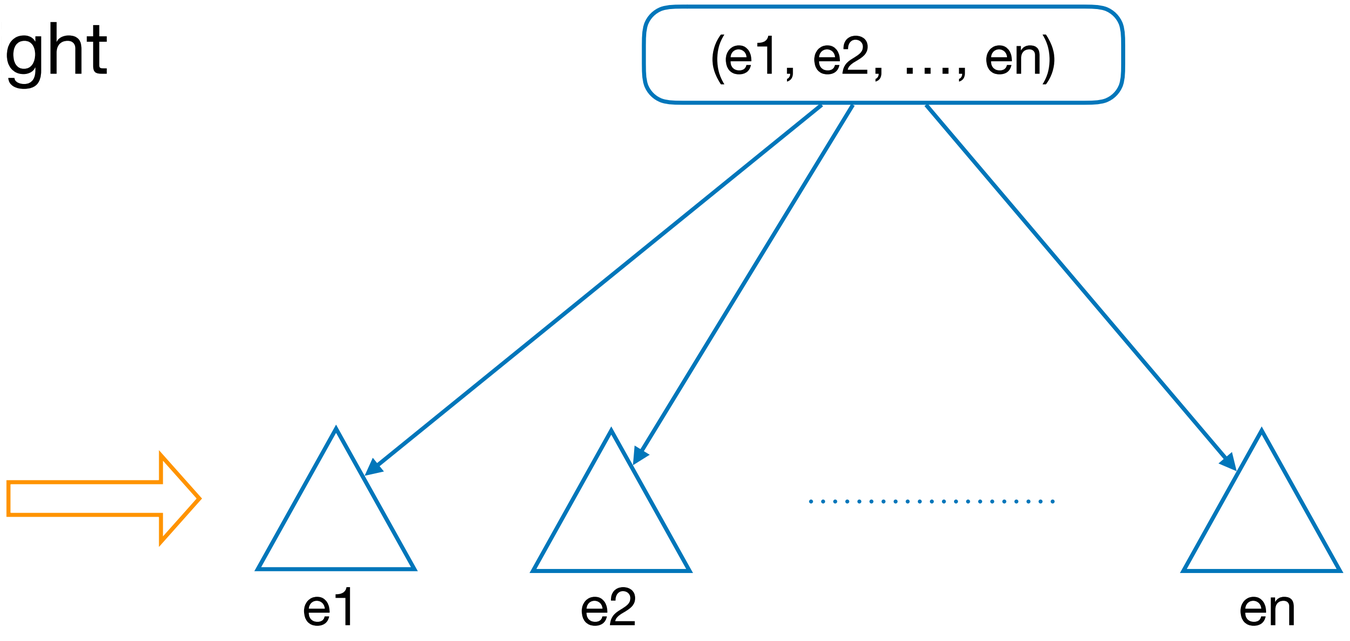
```
if e1 then e2
else e3
```

  - Let-expression:

```
let a = e1
and b = e2
and c = e3
in …
```

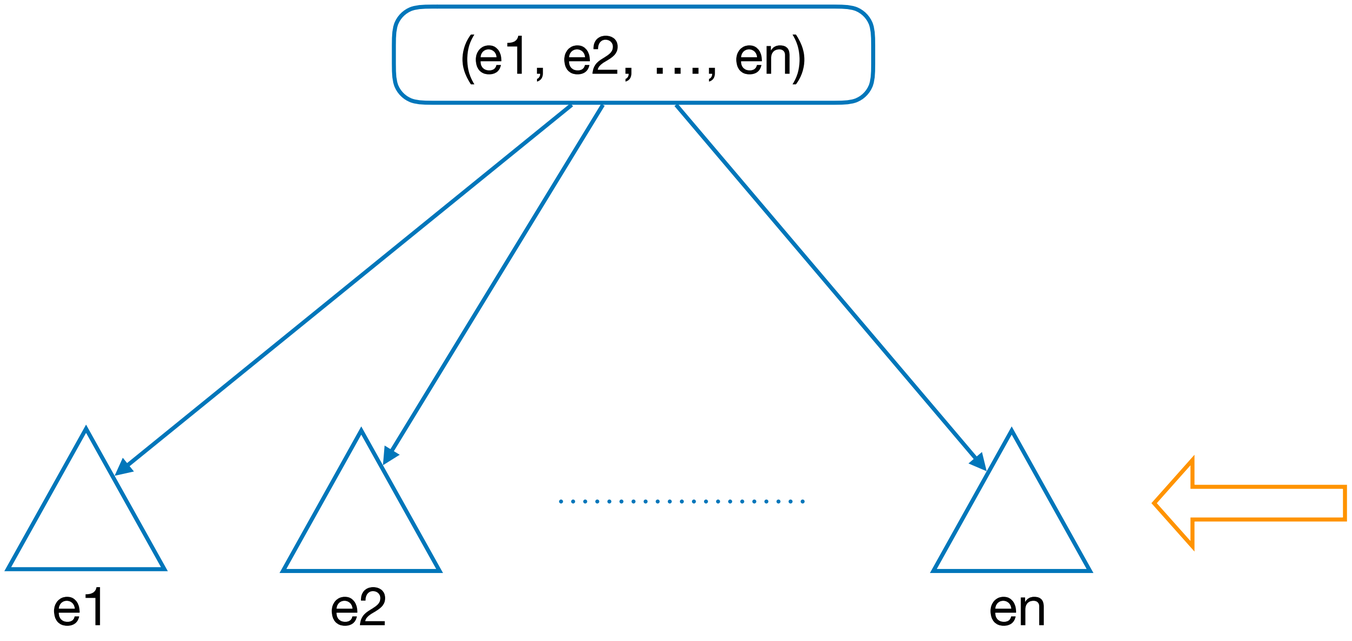
Type errors have a way

- Illustration: type-checking a tuple (e1, e2, …, en)
- Left to right



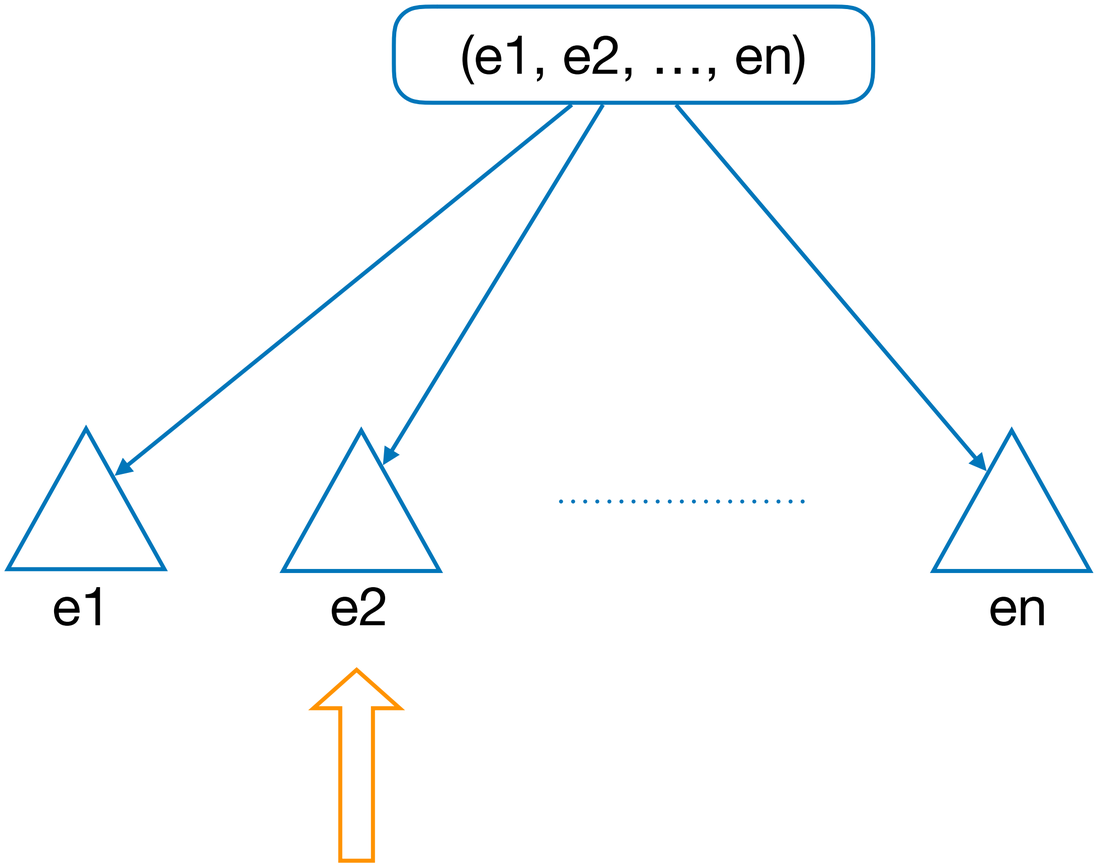
Type errors have a way

- Illustration: type-checking a tuple (e1, e2, …, en)
- Right to left



Type errors have a way

- Illustration: type-checking a tuple (e1, e2, …, en)
- Random order



Implementation

- Implemented on top of the OCaml compiler 4.08. Main module: ocaml/typing/typecore.ml
- GitHub repository: https://github.com/type-errors/ocaml
- Change the default behavior of OCaml compiler to report multiple type errors:
  - Using the flag: `-type-errors`

Implementation

- To parameterize the traversal order:
  - Delay the type-checking of sub-terms using thunks.
  - Control the traversal order by flags:
    - `-left-to-right`
    - `-right-to-left`
    - `-random-order`

Illustration: if-then-else expression

- Thunk-ify the processing of the “then” and the “else” branches

```
| Pexp_ifthenelse(scond, sifso, sifnot) ->
    let cond = type_expect env scond … in
    begin match sifnot with
    | None -> …
    | Some sifnot ->
-       let ifso = type_expect env sifso ty_expected in
-       let ifnot = type_expect env sifnot ty_expected in
+       let thunk1 () = type_expect env sifso ty_expected in
+       let thunk2 () = type_expect env sifnot ty_expected in
+       let ifso, ifnot = apply_thunk_pair thunk1 thunk2 in
        unify_exp env ifnot ifso.exp_type;
        re {
          exp_desc = Texp_ifthenelse(cond, ifso, Some ifnot);
          … }
```

Illustration: if-then-else expression

- Supporting function: `apply_thunk_pair`

```
let apply_thunk_pair thunk1 thunk2 =
  let apply_left_to_right () =
    let e1 = apply_thunk thunk1 in
    let e2 = apply_thunk thunk2 in
    (e1, e2) in
  let apply_right_to_left () =
    let e2 = apply_thunk thunk2 in
    let e1 = apply_thunk thunk1 in
    (e1, e2) in
  match get_type_infer_order () with
  | LeftToRight -> apply_left_to_right ()
  | RightToLeft -> apply_right_to_left ()
  | RandomOrder ->
      let _ = Random.self_init() in
      if Random.bool() then apply_left_to_right ()
      else apply_right_to_left ()
```

Illustration: tuple expression

- Thunk-ify the processing of sub-expressions in a tuple

```
| Pexp_tuple sexpl ->
    assert (List.length sexpl >= 2);
    let subtypes = List.map (fun _ -> newgenvar ()) sexpl in
    let to_unify = newgenty (Ttuple subtypes) in
    with_explanation (fun () ->
      unify_exp_types loc env to_unify ty_expected);
-   let expl = List.map2 (fun body ty ->
-       type_expect env body (mk_expected ty)) sexpl subtypes in
+   let thunk_expl = List.map2 (fun body ty ->
+       (fun () -> type_expect env body (mk_expected ty))) sexpl subtypes in
+   let expl = apply_thunk_list thunk_expl in
    re {
      exp_desc = Texp_tuple expl;
      exp_type = newty (Ttuple (List.map (fun e -> e.exp_type) expl));
      ...}
```

Illustration: tuple expression

- Supporting function: `apply_thunk_list`

```
let apply_thunk_list thunks =
  match get_type_infer_order () with
  | LeftToRight ->
      List.map apply_thunk thunks
  | RightToLeft ->
      List.rev_map apply_thunk (List.rev thunks)
  | RandomOrder ->
      let thunks, indices = shuffle_list thunks in
      let results = List.map apply_thunk thunks in
      unshuffle_list results indices
```

Experiment

Example 1: integer expressions.

- Multiple errors
- Left-to-right order

```
(* program successor2.ml *)

let f x =
  (succ y x, y)
```

```
trungtq@bb8 $ ../../ocamlc successor2.ml -type-errors -left-to-right
File "successor2.ml", line 5, characters 3-7:
5 |   (succ y x, y)
       ^^^^
Error: This function has type int -> int
       It is applied to too many arguments; maybe you forgot a `;'.


File "successor2.ml", line 5, characters 13-14:
5 |   (succ y x, y)
                 ^
Error: Unbound value y
```

Experiment

Example 1: integer expressions.

- Multiple errors
- Right-to-left order

```
1
2 (* program successor2.ml *)
3
4 let f x =
5   (succ y x, y)
6
7
```

```
trungtq@bb8 $ ../../ocamlc successor2.ml -type-errors -right-to-left
File "successor2.ml", line 5, characters 13-14:
5 |   (succ y x, y)
                ^
Error: Unbound value y


File "successor2.ml", line 5, characters 3-7:
5 |   (succ y x, y)
       ^^^^
Error: This function has type int -> int
       It is applied to too many arguments; maybe you forgot a `;'.
```

Experiment

Example 2: boolean expressions.

- Multiple errors
- Left-to-right order

```
(* program bool2.ml *)

let b =
  (not f true, f false)
```

```
trungtq@bb8 $ ../../ocamlc bool2.ml -type-errors -left-to-right
File "bool2.ml", line 5, characters 3-6:
5 |   (not f true, f false)
       ^^^
Error: This function has type bool -> bool
       It is applied to too many arguments; maybe you forgot a `;'.


File "bool2.ml", line 5, characters 15-16:
5 |   (not f true, f false)
                  ^
Error: Unbound value f
```

Experiment

Example 2: boolean expressions.

- Multiple errors
- Right-to-left order

```
(* program bool2.ml *)

let b =
  (not f true, f false)
```

```
trungtq@bb8 $ ../../ocamlc bool2.ml -type-errors -right-to-left
File "bool2.ml", line 5, characters 15-16:
5 |   (not f true, f false)
                  ^
Error: Unbound value f


File "bool2.ml", line 5, characters 3-6:
5 |   (not f true, f false)
       ^^^
Error: This function has type bool -> bool
       It is applied to too many arguments; maybe you forgot a `;'.
```

Experiment

Example 3: string expressions

- Multiple errors
- Left-to-right order

```
1
2 (* program string2.ml *)
3
4 let f g z t =
5   (g (z "a"), z ^ "a", z == "a")
6
```

```
trungtq@bb8 $ ../../ocamlc string2.ml -type-errors -left-to-right
File "string2.ml", line 5, characters 14-15:
5 |   (g (z "a"), z ^ "a", z == "a")
                 ^

Error: This expression has type string -> 'a
       but an expression was expected of type string


File "string2.ml", line 5, characters 28-31:
5 |   (g (z "a"), z ^ "a", z == "a")
                               ^^^

Error: This expression has type string but an expression was expected of type
         string -> 'a
```

Experiment

Example 3: string expressions

- Multiple errors
- Right-to-left order

```
1
2 (* program string2.ml *)
3
4 let f g z t =
5   (g (z "a"), z ^ "a", z == "a")
6
```

```
trungtq@bb8 $ ../../ocamlc string2.ml -type-errors -right-to-left
File "string2.ml", line 5, characters 6-7:
5 |   (g (z "a"), z ^ "a", z == "a")
          ^
Error: This expression has type string
       This is not a function; it cannot be applied.
```

Experiment

Example 3: string expressions

- Multiple errors
- Random order (1)

```
1
2 (* program string2.ml *)
3
4 let f g z t =
5   (g (z "a"), z ^ "a", z == "a")
6
```

```
trungtq@bb8 $ ../../ocamlc string2.ml -type-errors -random-order
File "string2.ml", line 5, characters 6-7:
5 |   (g (z "a"), z ^ "a", z == "a")
          ^
Error: This expression has type string
       This is not a function; it cannot be applied.
```

Experiment

Example 3: string expressions

- Multiple errors
- Random order (2)

```
1
2 (* program string2.ml *)
3
4 let f g z t =
5   (g (z "a"), z ^ "a", z == "a")
6
```

```
trungtq@bb8 $ ../../ocamlc string2.ml -type-errors -random-order
File "string2.ml", line 5, characters 14-15:
5 |   (g (z "a"), z ^ "a", z == "a")
                 ^
Error: This expression has type string -> 'a
       but an expression was expected of type string


File "string2.ml", line 5, characters 28-31:
5 |   (g (z "a"), z ^ "a", z == "a")
                               ^^^
Error: This expression has type string but an expression was expected of
         string -> 'a
```

Experiment

Example 3: string expressions

- Multiple errors
- Random order (3)

```
(* program string2.ml *)

let f g z t =
  (g (z "a"), z ^ "a", z == "a")
```

```
trungtq@bb8 $ ../../ocamlc string2.ml -type-errors -random-order
File "string2.ml", line 5, characters 28-31:
5 |   (g (z "a"), z ^ "a", z == "a")
                             ^^^
Error: This expression has type string but an expression was expected of
         string -> 'a


File "string2.ml", line 5, characters 14-15:
5 |   (g (z "a"), z ^ "a", z == "a")
                 ^
Error: This expression has type string -> 'a
       but an expression was expected of type string
```

Related work

- McAdam's thesis (Repairing Type Errors in Functional Programs — Uni. Edinburgh 2002) states that some traversal orders elicit more meaningful type error messages than others.
- Pottier and Rémy (The Essence of ML Type Inference — MIT Press 2005) proposed to stage type inference into generating constraints and solving constraints to diagnose errors.
- Neubauer and Thiemann (Discriminative sum types locate the source of type errors — ICFP 2003) extended the type system to capture all the type errors in one derivation.
- None of them propose to alternate the traversal orders on demand as we do.

Type errors at IFL'19

1. Type debugging with counter-factual type error messages using an existing type checker

— Kanae Tsushima, Olaf Chitil, and Joanna Sharrad

> "Finally, we want to develop heuristics for our type debugger. Which potential type error location should the debugger ask about first? Are some locations more likely to be correct than others? Currently we have no answers to these questions." (Section 9).

Our analysis: having alternative error messages directly synergizes with their type debugger.

Type errors at IFL'19

2. Attribute grammars fly first-class... safer!

— Juan García Garland, Alberto Pardo, and Marcos Viera

> "To get clear error messages we had to deal with some tradeoffs. It requires careful management of context information annotated in types, and explicit term level proxy arguments to carry type information during type checking." (Section 6).

Our analysis: no careful management is needed if all error messages are enumerated. The users know best.

Type errors at IFL'19

3. A new view on parser combinators

— Pieter Koopman and Rinus Plasmeijer

> "A disadvantage of a tagless DSL is that the type error messages can be somewhat intimidating. It requires some training to understand them quickly." (Section 10).

Our analysis: having alternative error messages looks useful
(cf. Juan García Garland et al.'s type-error message with 8000 lines).

Conclusion

- A type system is declarative and it does not convey anything meaningful for an ill-typed program.
- The type system is operationalized into an implementation:
  - It emits type errors based on traversal order over sub-terms.
  - The reported type errors might not be useful for users.
- We propose to parameterize type inferencers with their traversal strategies:
  - More than one strategy (L2R, R2L, random) can be used over ill-typed terms.
  - A wider variety of error messages can be emitted on demand.

Our rationale to take home with you

- By force, each of us has developed diagnostic expertise to understand type errors.
- Our proposal is the only one that both:
  - solely capitalizes on this diagnostic expertise, and
  - synergizes with (a very simple adaptation of) existing type inferencers.

Thanks for your attention.